

Telegram_Restricted_Media_Downloader

A telegram downloader on Windows and Linux platform based on Python.

Python 3.13.2 | pyrogram@kurigram 2.2.26 | Platform Windows & Linux | DeepWiki Documentation

> [!NOTE]
> 由于本项目**提供**的Linux版本可能对较早版本的Linux系统兼容性较差。
> 若**无法运行的**Linux用户请**阅读**:*"3.0.在生产环境中运行（对于Linux用户）"*或*"6.0.通过Docker运行"*。
> 如果你**遇到任何问题**，请先仔细**阅读**:*"常见问题及解决方案汇总"*。
> **没有找到解决方案**再进群或私聊提问。

免责声明

本项目以`GNU GPL-3.0`协议开源发布。

所有使用本软件的行为及其后果均由使用者自行承担全部法律责任，开发者不对任何使用行为及其后果负责。

本项目按"原样"提供，不附带任何明示或暗示的保证。

作者	Gentlesprite
YouTube视频教程	点击观看
Telegram交流群	点击加入
支持作者	点击跳转

1.0.下载地址

平台	下载地址	备注
Github	点击跳转	–
Gitee	点击跳转	仅发布源码
蓝奏云	点击跳转	密码:ceze
Telegram交流群	点击加入	群文件

1.1.推荐终端（选看）

▼ **点击展开**

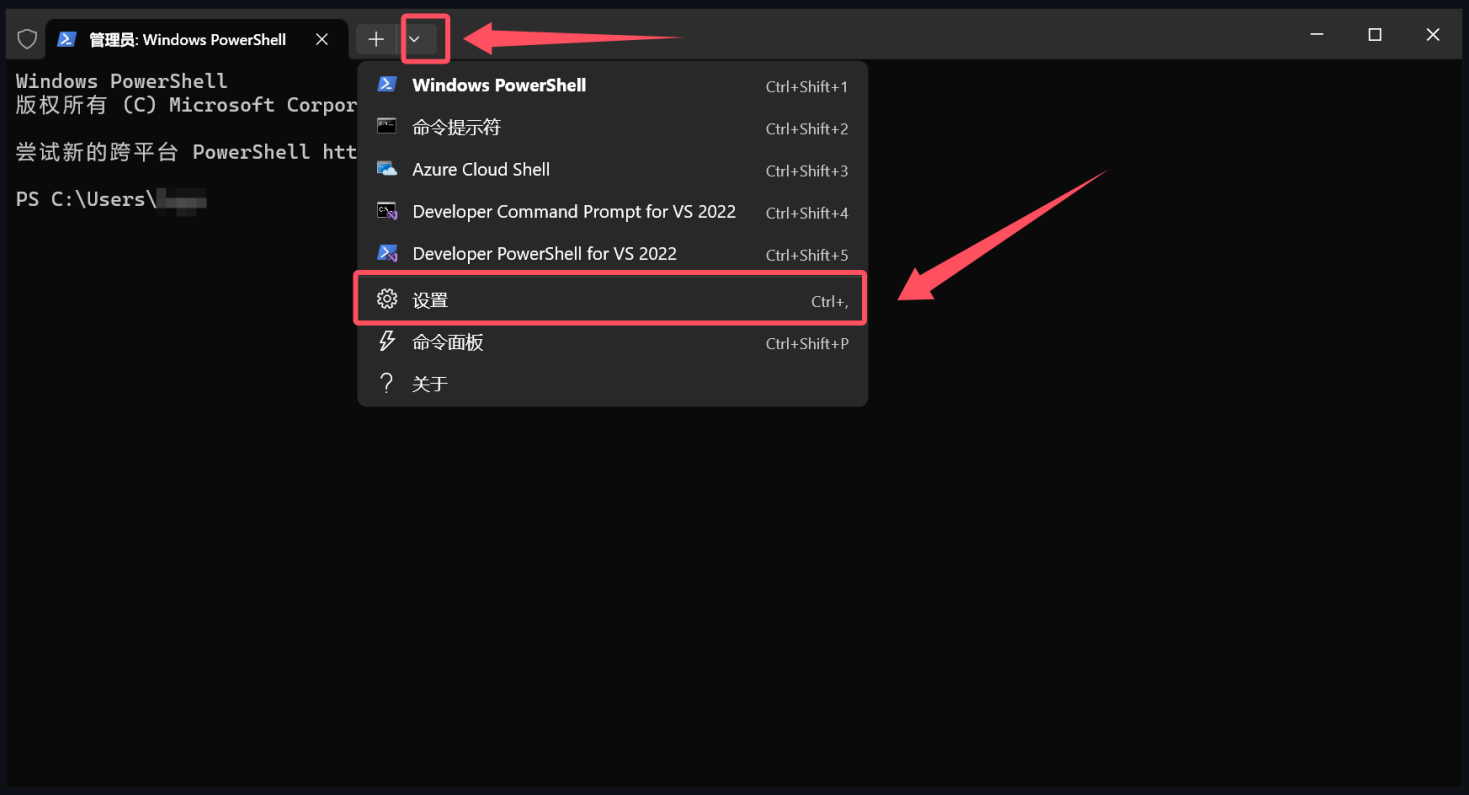
1. *对于Windows11用户*，`Windows Terminal`默认**已经安装好**，可**跳过**下载的步骤，直接前往第3步（将`Windows Terminal`设为**默认终端**）。

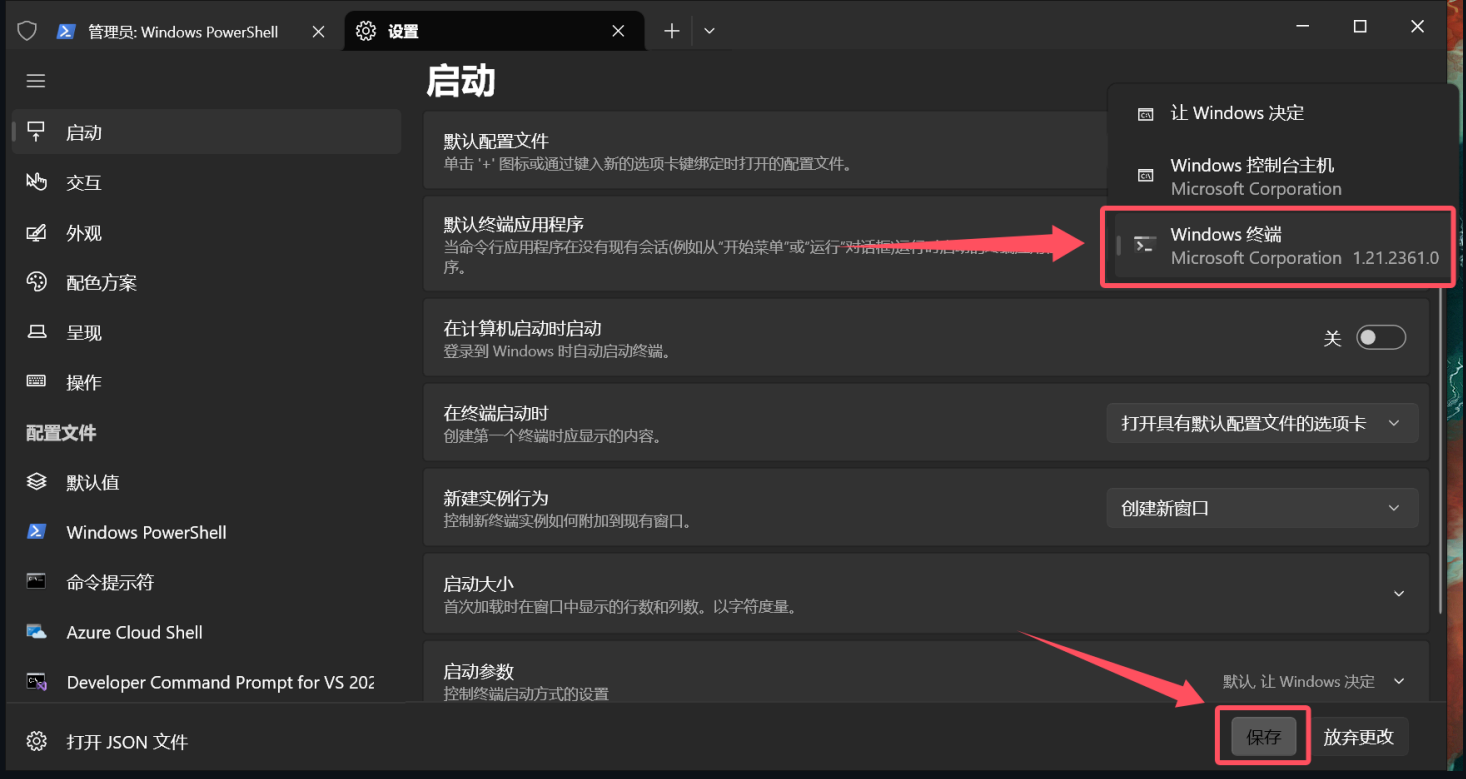
2. *对于Windows10用户*，推荐使用`Windows Terminal`作为**默认终端**，仅作为推荐安装，无论安装与否**不会影响本软件的使用**，`Windows Terminal`能提供更出色的显示、交互、体验效果，以及避免出现**文字显示**乱码。

   `Windows Terminal`下载地址如下表：

平台	下载地址
微软商店	点击跳转
Github	点击跳转

3. 下载完成完成后`win+r`输入`wt`回车打开，然后将`Windows Terminal`设为**默认终端**再启动软件，教程如下：

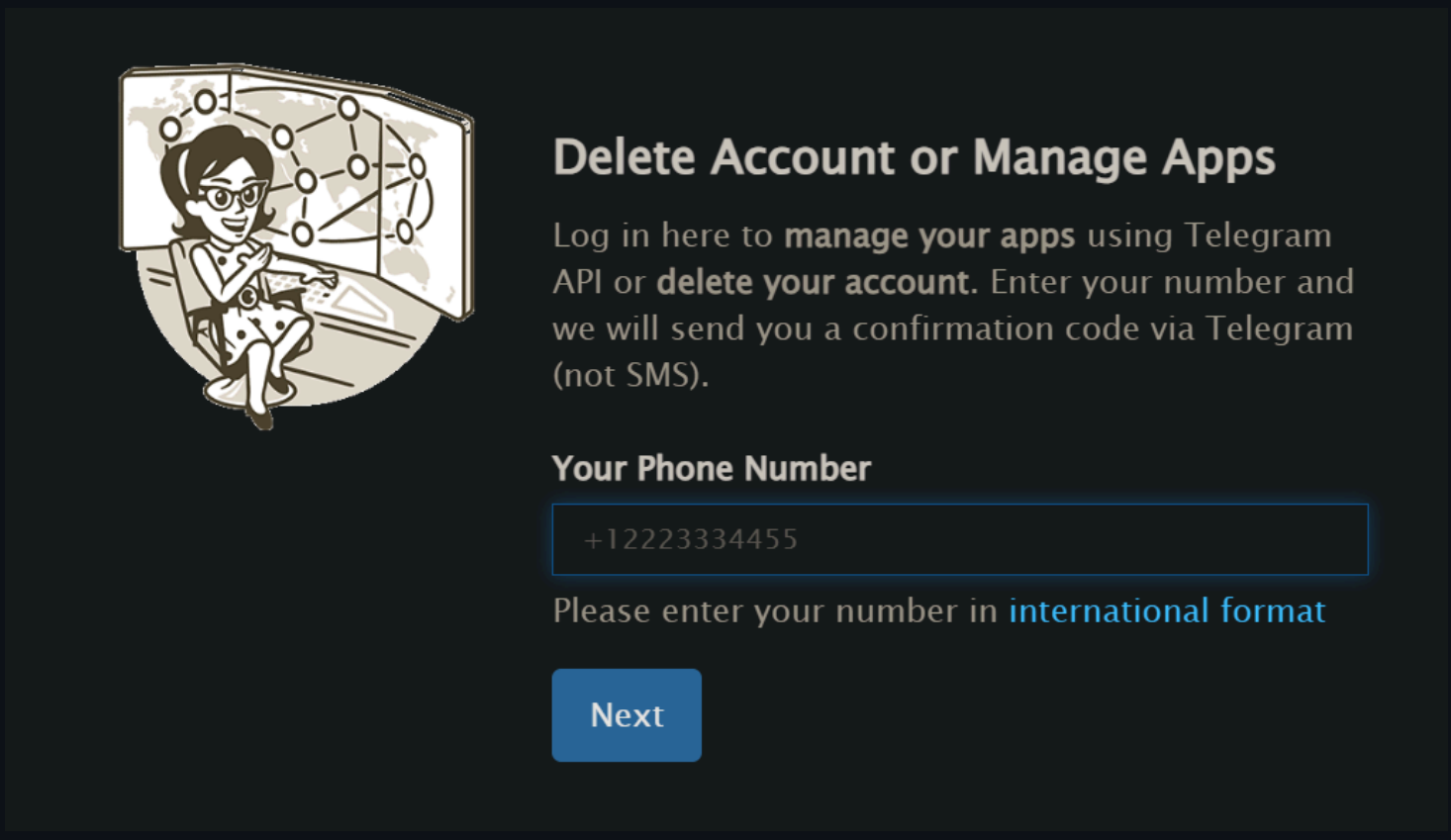




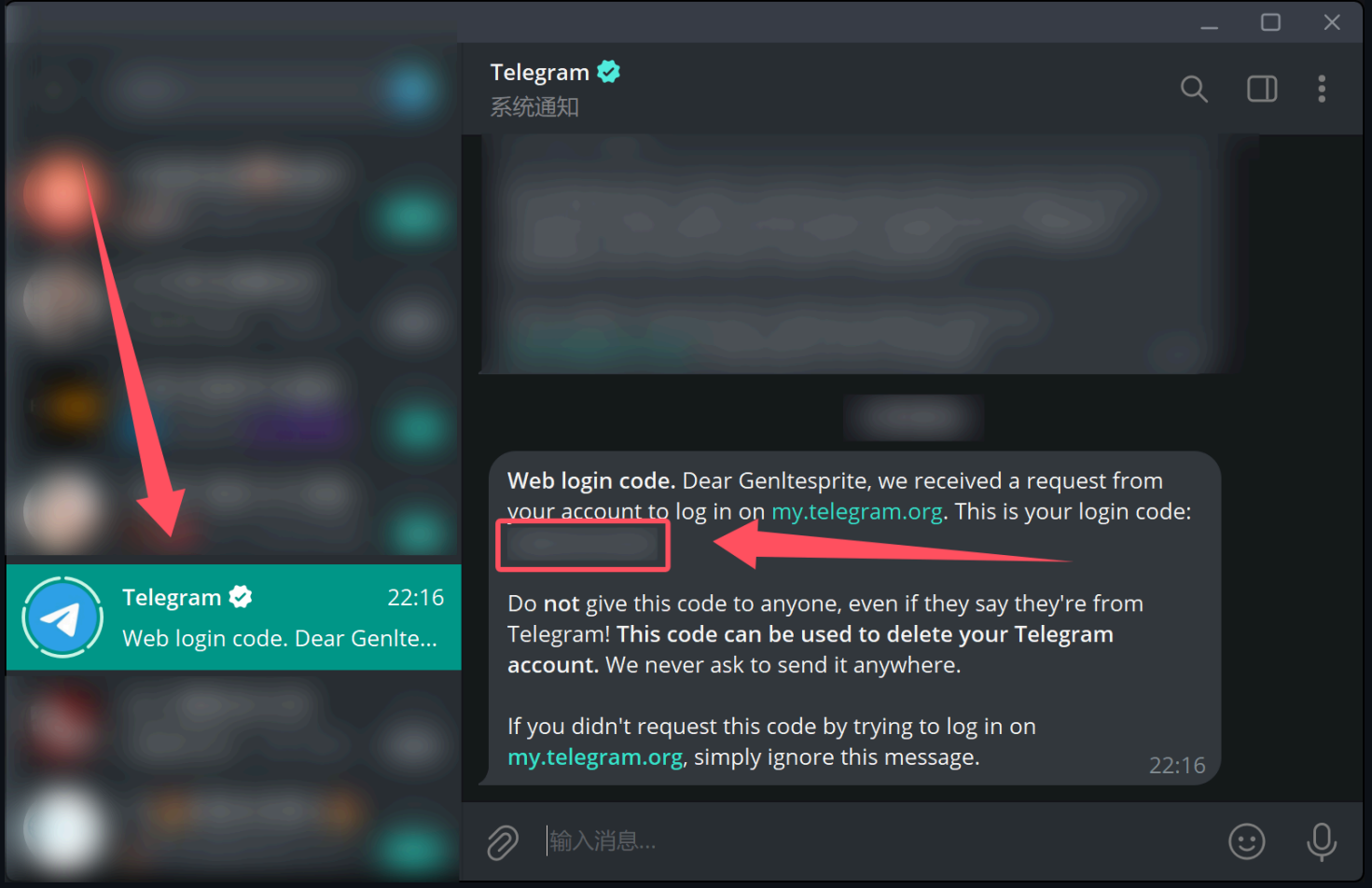
2.0.快速开始

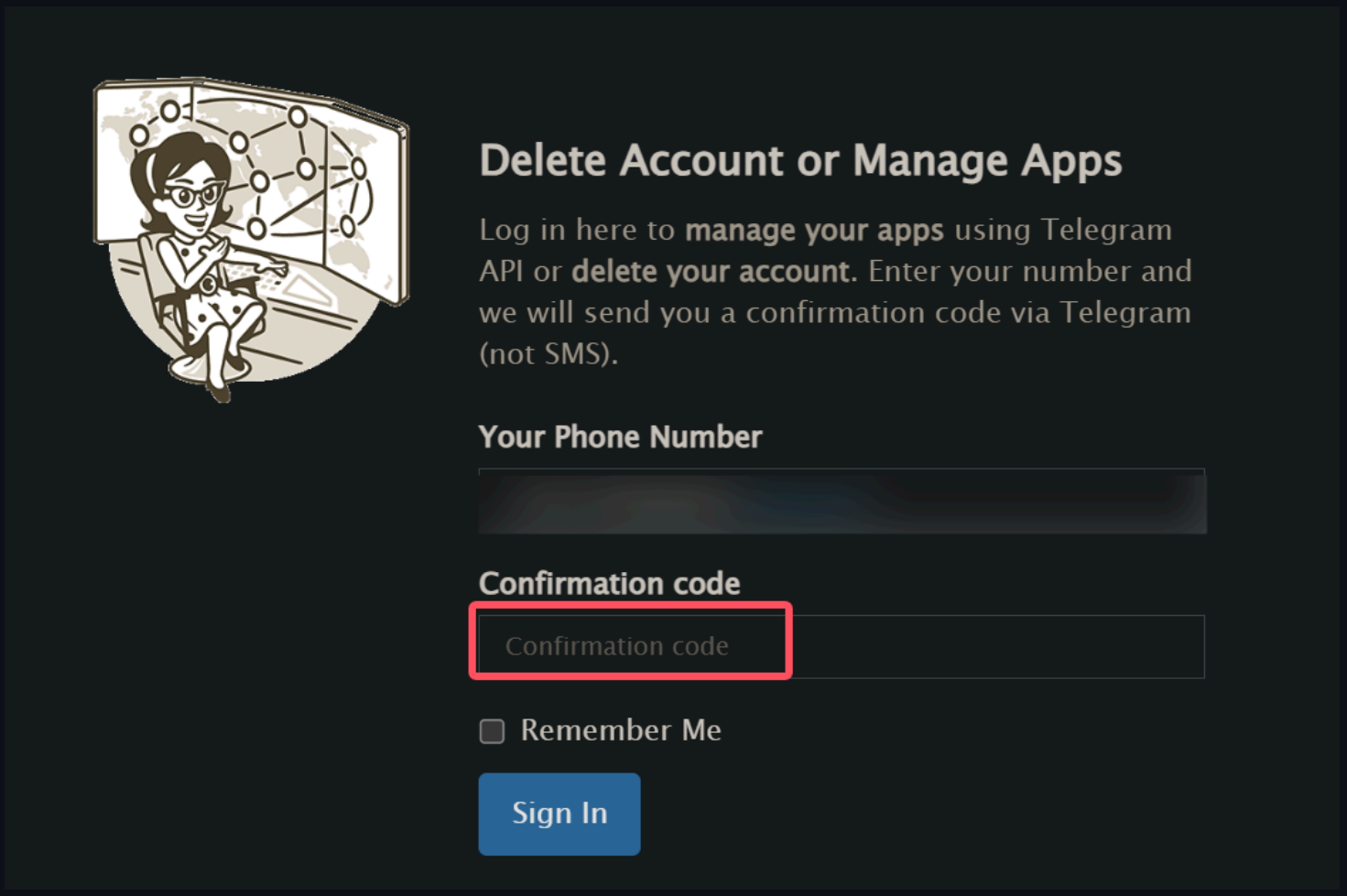
2.1.申请电报API

1. 前往网站:https://my.telegram.org/auth
2. 填写**自己绑定**`Telegram`电报的**手机号**注意手机号格式先要+地区再写入电话号码例如`+12223334455`，`+1`为地区，`222333445`为你绑定`Telegram`的**手机号**，填写后点击`Next`。

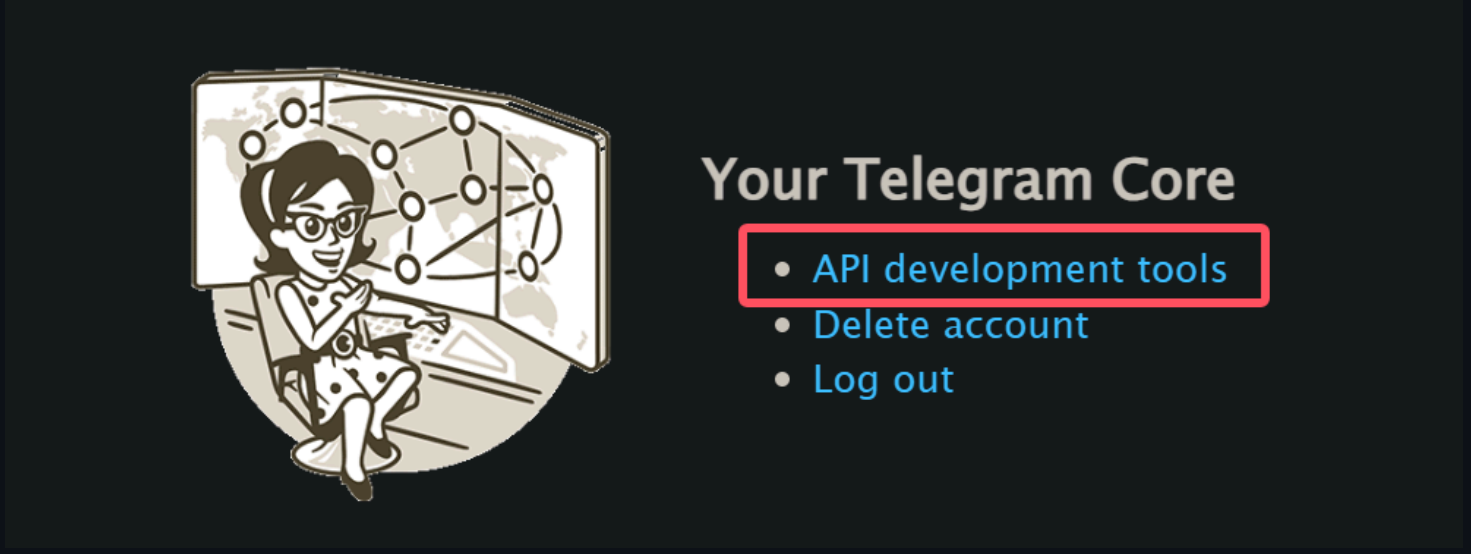


3. 打开你的`Telegram`客户端，此时会收到来自`Telegram`账号的消息，将上面的验证码填入`Confirmation code`框中，然后点击`Sign in`。





4. 点击`API development tools`按照提示填入即可。



5. 申请成功会得到一个`api_hash`和`api_id`保存下载，**切记不要泄露给任何人！**

2.2.电报机器人`bot_token`申请及使用教程（选看）

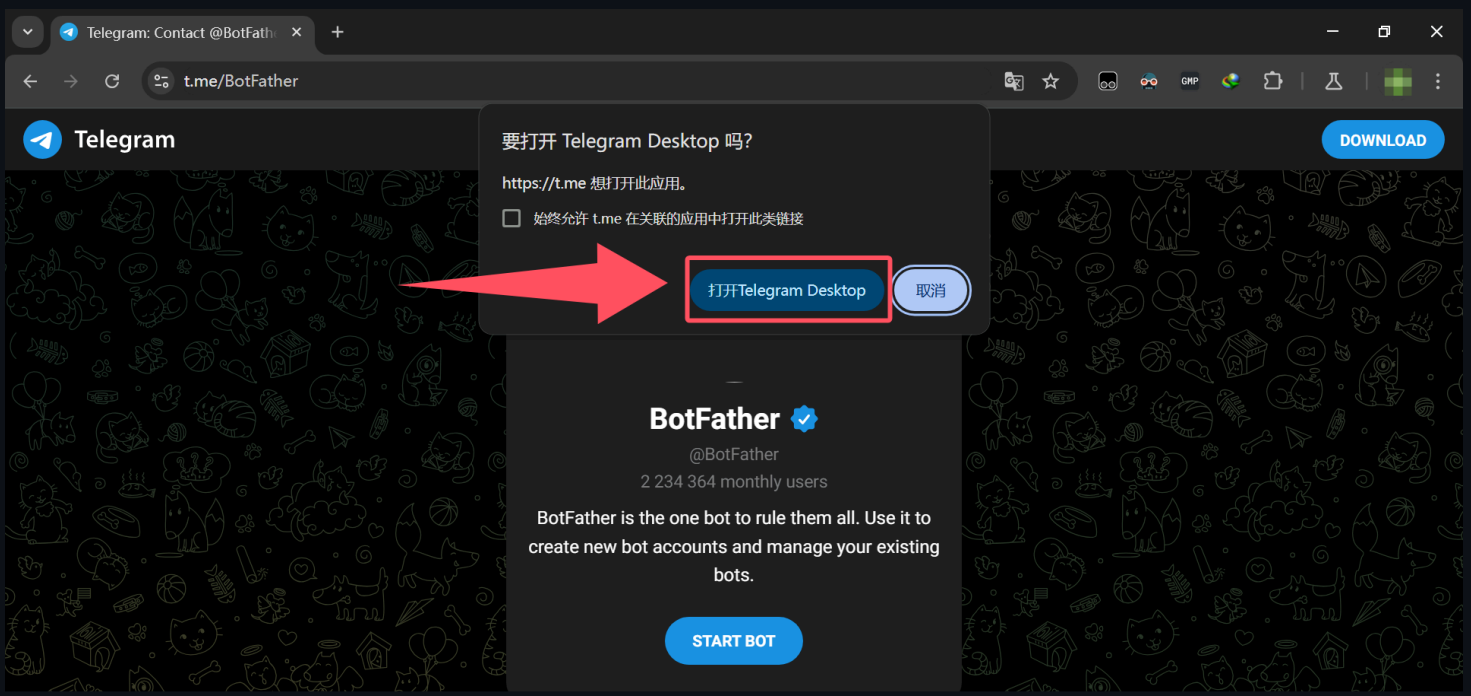
> [!NOTE]
> 如果配置了机器人，只要**保持软件运行**，就能实现**多端发送下载命令**并且**随时进行下载**。
> 故可以将软件部署在服务器上，无论是Windows还是Linux平台。
> Windows平台可直接使用releases里发布的二进制文件放在服务器运行。
> Linux平台的部署教程请**阅读**:*"3.0.在生产环境中运行（对于Linux用户）"*。

▼ **点击展开**

2.2.1.申请教程

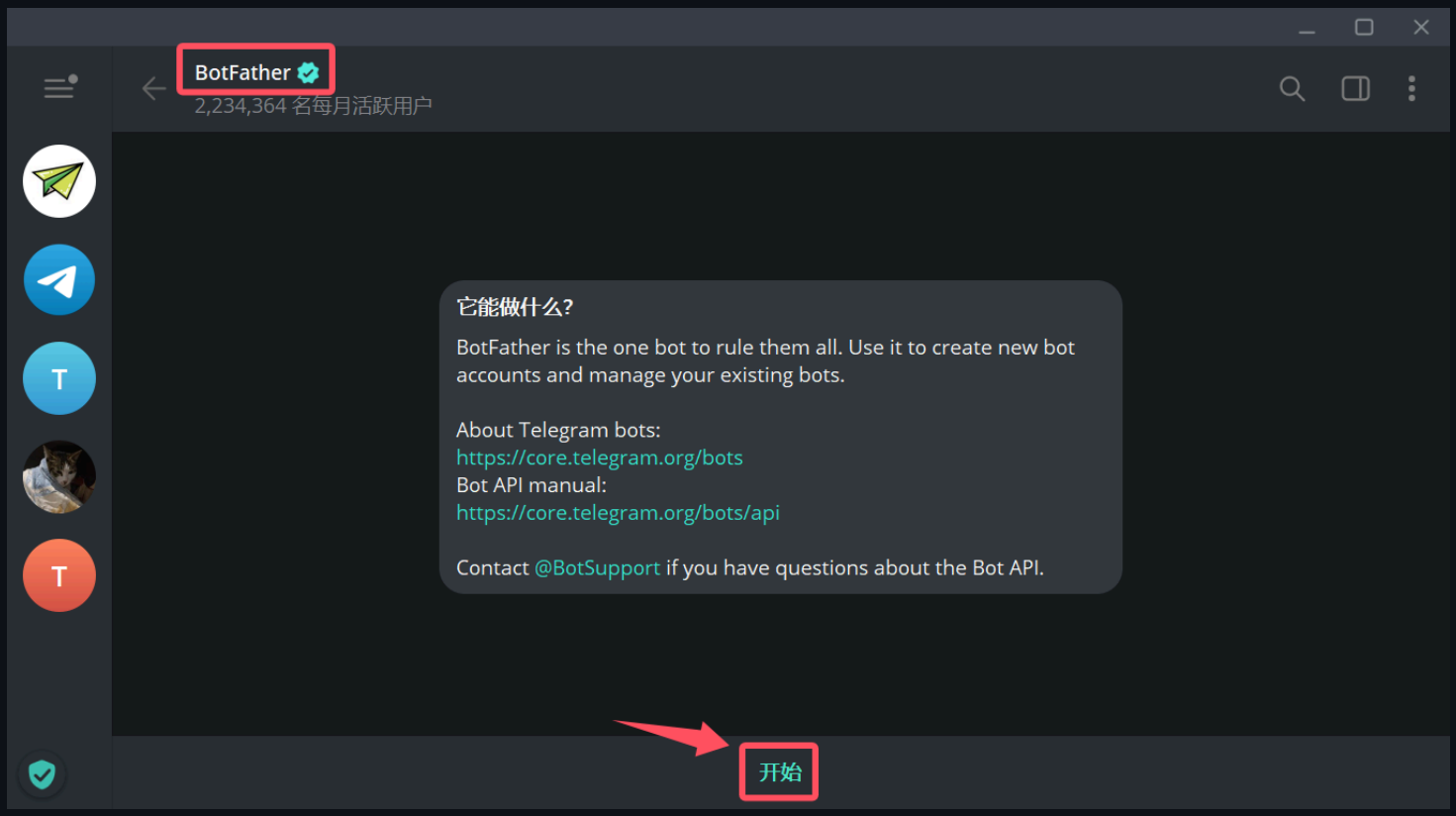
1. 前往网站：https://t.me/BotFather

2. 打开后会**提示**"要打开 Telegram Desktop 吗？"此时**点击**"打开Telegram Desktop"如下图所示：

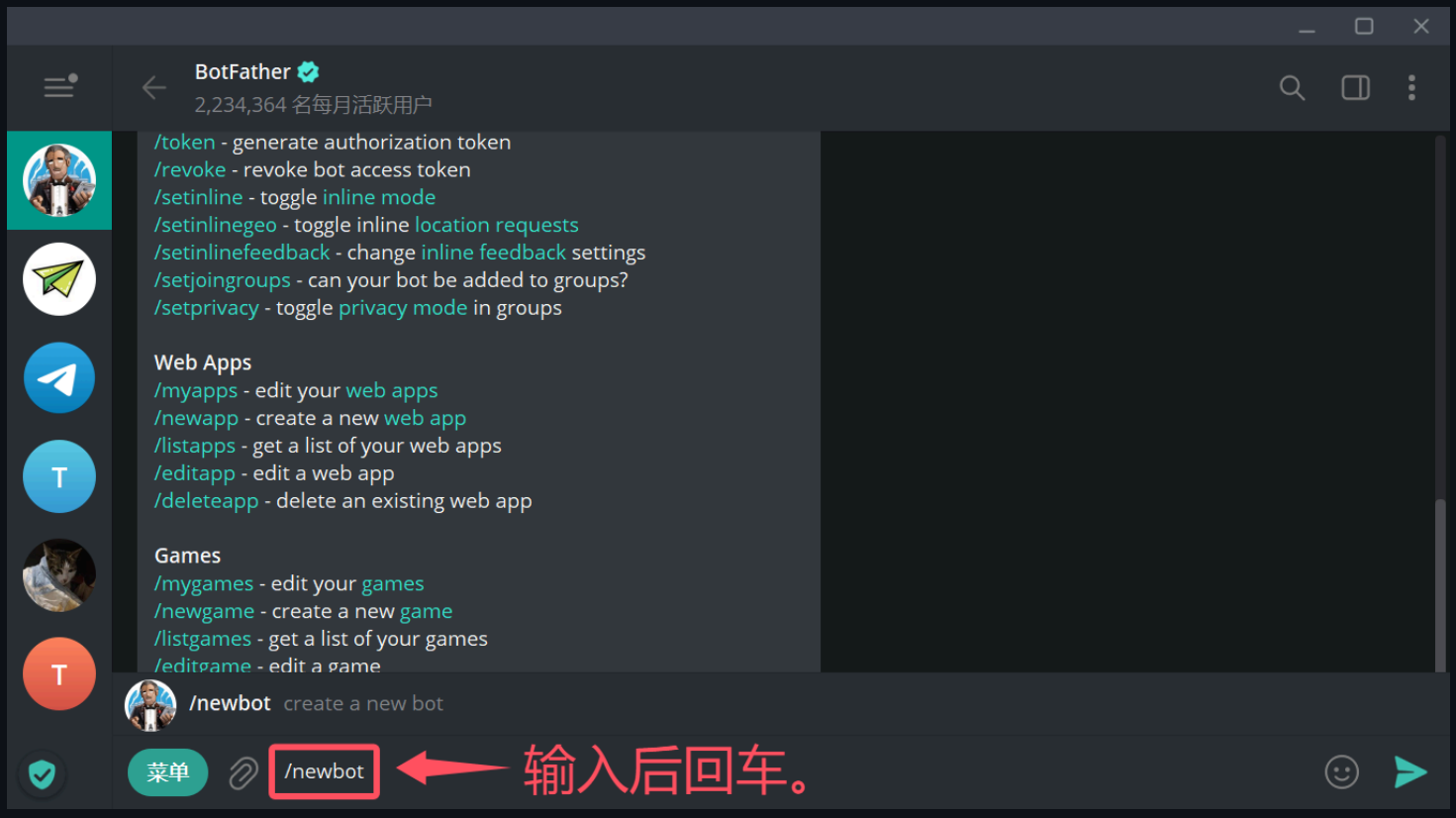


如果没有这个弹窗，说明电脑没有安装Telegram**客户端**，安装后再重试即可。

3. **点击开始**，如下图所示：



4. 然后在当前**聊天框**中输入`/newbot`后回车，如下图所示：



它会回复你`"Alright, a new bot. How are we going to call it? Please choose a name for your bot."`意思是给机器人取一个名字，如下图所示：

5. 这个名字是显示名称（display name），并不是唯一识别码，随便设置一下即可，之后可以通过`/setname`命令进行修改。

6. 接着设置机器人的**唯一名称**。字符串必须 以`bot`结尾，比如 `HelloWorld_bot` 或 `HelloWorldbot` 都是合法的。如果设置的名字已经被占用需要重新设置。如设置成了 `trmd_bot` 但是这个名字已经有人使用了，此时会提示`"Sorry, this username is already taken. Please try something different."`意思是已经被使用了，需要拟定一个**不重复**的，如下图所示：

   如果结果如**上图**所示，则就代表名字**重复**了，需要**重新拟定**一个。

7. 直到提示你`"Done! Congratulations on your new bot. . ."`如下图所示：

   如果结果如**上图**所示，则代表`bot_token`申请成功了，箭头指的红框处就是你所申请的`bot_token`，**切记不要泄露给任何人！**

2.2.2.使用教程

1. 申请完成后，在软件配置时询问"是否启用「机器人」（需要提供bot_token）？ - 「y|n」（默认n）"选择`y`代表**需要**使用，如下图所示：

   然后在**上图**箭头所指处填入"2.2.1.申请教程"第7步申请的`bot_token`后回车，即可配置完成。

2. 在一切配置完成，软件启动成功后等待提示"「机器人」启动成功。"，就代表机器人可以使用了，如下图所示：

3. 在`Telegram`客户端中找到与`BotFather`的对话框，找到"2.2.1.申请教程"第7步对话的位置（或者用你自己的方式找到你的机器人的对话框），如下图所示：

   然后在**上图**箭头所指处即可**跳转**到机器人对话框。

4. **点击**开始，如下图所示：

   不出意外，会收到一条来自**机器人**发送的消息，如下图所示：

   如果没收到尝试给机器人发送任意命令。

5. 目前机器人支持的命令用法及解释如下表所示：

命令	用法	解释
`/help`	向机器人发送发送`/help`即可。	展示**可用**命令。
`/download`	`/download 链接1 链接2 链接3 链接n` 或`/download 频道链接 1 100`	分配**新的**下载任务，两种方式可选（**指定链接下载**和**范围下载**，具体使用方法请见下方说明）。
`/table`	向机器人发送`/table`即可。	在**终端**输出**当前**下载情况的**统计信息**。
`/forward`	`/forward https://t.me/A https://t.me/B 1 100`	将**频道A**的消息转发至**频道B**，其中`1`代表`起始ID`，`100`代表截止`ID`。
`/exit`	向机器人发送`/exit`即可。	**退出**软件。
`/listen_download`	`/listen_download https://t.me/A https://t.me/B https://t.me/n`	**实时**监听**频道A**、**频道B**和**频道n**的**最新消息**（视频和图片）进行下载。
`/listen_forward`	`/listen_forward https://t.me/A https://t.me/B`	**实时**监听**频道A**的**最新消息**（任意消息）转发至**频道B**。但当**频道A**为**私密频道**时候无法转发。
`/listen_info`	向机器人发送`/listen_info`即可。	查看当前已经创建的监听信息。
`/upload`	`/upload` `本地文件` `目标频道`	上传**本地的文件**到**指定频道**。
`/upload_r`	`/upload_r 本地文件夹 目标频道`	**递归**上传文件夹（**包含子文件夹**）到**指定频道**。
`/download_chat`	`/download_chat 频道链接`	下载**指定频道**并支持**通过内联键盘自定义内容过滤**。

其他功能：

- （`≥v1.8.7`）转发`视频`、`图片`、`音频`、`语音`、`GIF`、`文档`、`视频笔记`、`实况图片`类型的消息给机器人，即可创建下载任务。
  - 此项功能不受用户自定义下载类型限制，确保文件即时获取。
  - 转发的消息将按新消息处理，每次均生成独立文件命名。暂不支持重复文件识别，请妥善管理多次转发的相同内容。
  - 此功能仅用于便利用户日常使用，对于无法被转发、下载的消息，请根据实际需求使用对应的命令。

2.2.3.命令

/help

菜单

/download

> ⚠️ **版本变更说明：**

版本	说明
`≥v1.6.3`	已全面支持下载时的断点续传功能（支持所有上传场景），增强了在较差网络环境下的传输稳定性与可靠性。

- 方式一：
  -
  - 只要发送了正确的下载命令，终端就会创建对应的下载任务，如下图所示：
  -
- 方式二：
  - 该功能为版本`≥v1.5.1`的新增功能，用于对于**单一链接**的范围下载，并且该方式要求指定**频道链接**、**起始ID**与**结束ID**：

```
# 语法格式如下：
/download 频道链接 起始ID 结束ID
# 举例：
/download https://t.me/test 1 500
# 代表下载https://t.me/test从消息ID=1到结束ID=500的媒体。
```

/table

需要**注意**的是，这个表格是**实时**的**状态**，并不是**最终**下载完成的**结果**，每一次使用它都会随着**当前**的**下载记录**而更新。

链接统计表的使用，如下图所示：

注意：由于早期代码设计缺陷，**链接统计表**为后续支持的功能：

- **链接统计表**仅会统计**所有支持的类型**，**并不会只统计**用户**当前所选择**的类型。
- **链接统计表**对于**评论区媒体**的统计，会出现**总数统计错误**的问题，体现在**总数**为`1`，**小于**当前的**下载数**，**完成率**`>>100%`的问题（该问题已在`≥v1.5.9`修复）。
- 当用户**未选择**下载**所有支持的类型**时，在**用户所选择的类型**下载完成后（或使用机器人发送**链接统计表**），尽管所有用户指定类型的文件已经下载完成，当**链接统计表**显示`完成率`不为`100%`时，代表该链接还存在其他用户未指定的文件类型，但实际用户所指定的类型已经下载完成了，是正常情况（该问题已在`≥v2.0.2`调整：在机器人设置或配置文件中被定义为跳过的下载类型与不支持的下载类型，统计时不再计入总数）。
- 版本`≥v1.6.5`起已支持**导出表格**功能，通过该命令可在运行时控制**是否**在退出后**导出指定类型的表格**。

计数统计表的使用，如下图所示：

/forward

> ⚠️ **注意：**
> 消息能否转发，在于频道是否开启了`限制保存内容`功能。
> 如果**无法转发，机器人**会在**聊天框**提供一个**下载按钮**与**下载后上传按钮**（`≥v1.6.7`）。

> ⚠️ **版本变更说明：**

版本	说明
`≥v1.6.9`	`/forward`将支持过滤转发类型。可通过`[帮助页面]`->`[设置]`->`[转发设置]`进行修改。
`≥v1.7.5`	为确保"受限转发"功能顺利完成，在**下载后上传**过程中，创建下载任务时将**忽略配置文件中设置的下载类型限制**，此时（指"受限转发"情况）`/forward`命令**无法按照**`[转发设置]`过滤类型。
`≥v1.8.5`	为确保"受限转发"功能顺利完成，在**下载后上传**过程中，创建下载任务时将**忽略该链接是否被重复添加的判定**。

- 转发消息语法：

```
/forward 频道A 频道B 起始ID 结束ID
```

- 实例：
  - 该实例代表转发`https://t.me/test`**频道**中从`消息ID=1`到`结束ID=500`的消息到`https://t.me/test2`**频道**。

```
/forward https://t.me/test https://t.me/test2 1 500
```

- 转发至个人收藏夹（任选一种命令即可）：
  - 方式1，使用`me`或`self`指向个人收藏夹（任选一种命令即可）：

```
/forward https://t.me/test me 1 500
```

```
/forward https://t.me/test self 1 500
```

  - 方式2，使用`https://t.me/用户名`：
    - 用户名即指个人账户信息里@后面那一串，需用户提前手动自己设置，否则无法使用该方式。
    - 例如通过查询个人信息，得知当前设置的用户名为`@developer`，此时指向个人的链接即为`https://t.me/developer`。

```
/forward https://t.me/test https://t.me/developer 1 500
```

  - 无论使用方式1或方式2，都代表转发`https://t.me/test`**频道 中从**`消息ID=1`**到结束**`ID=500`**的消息到个人收藏夹**。

- 转发个人收藏夹的内容至任意频道：
  - 方式1：

```
/forward me https://t.me/test 1 500
```

- 方式2：

```
/forward https://t.me/developer me 1 500
```

- 无论使用方式1或方式2，都代表转发**个人收藏夹**中从`消息ID=1`到结束`ID=500`的消息到`https://t.me/test`**频道**。

/exit

/listen_download

- `/listen_download`监听下载用于，实时监听该链接的最新消息进行下载。
  - 在用户发送了正确的监听命令后，会收到机器人的成功提示。
  - 当被监听的频道有可下载的内容时，就会自动发送命令下载。

注册监听下载

```
/listen_download https://t.me/A
```

注销监听下载

注销方法

再次向机器人发送创建监听时的命令，机器人将会提供给用户一个用于注销监听的内联键盘，点击确认即可。

与此同时，也可通过`网页面板`的`监听`版块提供的按钮注销监听下载（≥v2.0.2）。

```
/listen_download https://t.me/A
```

注册多个监听下载

```
/listen_download https://t.me/A https://t.me/B https://t.me/n
```

注销多个监听下载

```
/listen_download https://t.me/A https://t.me/B https://t.me/n
```

/listen_forward

> ⚠️ **版本变更说明：**

版本	说明
`≥v1.6.7`	当检测到"受限转发"时，自动采用"下载后上传"的方式（默认**开启**）。当**下载并完成上传**后，可选择**是否删除本地文件**（默认**关闭**）。并且可通过`[帮助页面]`->`[设置]`->`[上传设置]`进行修改。
`≥v1.6.9`	`/listen_forward`将支持过滤转发类型。可通过`[帮助页面]`->`[设置]`->`[转发设置]`进行修改。
`≥v1.7.5`	为确保"受限转发"功能顺利完成，在**下载后上传**过程中，创建下载任务时将**忽略配置文件中设置的下载类型限制**。
`≥v1.8.5`	为确保"受限转发"功能顺利完成，在**下载后上传**过程中，创建下载任务时将**忽略该链接是否被重复添加的判定**。

- `/listen_forward`监听转发用于，实时监听该链接的最新消息。
  - 与`/forward`命令一样，消息能否转发，在于频道是否开启了`限制保存内容`功能。
  - 但在`≥v1.6.7`起可以通过下载再上传的方式进行"转发"。
  - 在用户发送了正确的监听命令后，会收到机器人的成功提示。
  - 当被监听的频道有**任何**新内容时，就会自动转发至用户所指定的频道。

监听行为说明

- **频道独占原则：**
  - 每个频道**同一时间**只能激活一种监听模式（下载或转发）。
  - 对于`/listen_forward`命令，"同一频道"特指**被监听**的源频道。
  - 转发目标频道**不受此限制**，仍可通过`/listen_download`创建下载任务。
- **操作限制：**
  - 当`频道A`正在监听转发`频道B`时：
    - 可以**同时**在`频道B`设置监听下载。
    - 但`频道B`的监听下载，不会响应来自`频道A`的监听转发。
- **监听切换流程：**
  - 必须先通过**同一命令**（注册监听时的命令）来取消现有监听。
  - 然后才能创建**新的监听**事件。

注册监听转发

```
/listen_forward https://t.me/A https://t.me/B
```

注销监听转发

注销方法

再次向机器人发送创建监听时的命令，机器人将会提供给用户一个用于注销监听的内联键盘，点击确认即可。

与此同时，也可通过`网页面板`的`监听`版块提供的按钮注销监听转发（≥v2.0.2）。

```
/listen_forward https://t.me/A https://t.me/B
```

/listen_info

- `/listen_info`用于查看**当前**已经创建的**监听信息**，**直接发送**即可：

```
/listen_info
```

/upload

> ⚠️ **版本变更说明：**

版本	说明
`≥v1.7.9`	已全面支持上传时的断点续传功能（支持所有上传场景），增强了在较差网络环境下的传输稳定性与可靠性。
`≥v1.8.4`	`/upload` 命令支持 `me`、`self` 作为目标频道的参数（`me`、`self` 作为目标频道参时，代表指向个人收藏夹 `Saved Messages`）。

- `/upload` 用于上传本地的文件到指定频道。
  - 支持上传文件夹（当指定路径为文件夹时并且版本需≥v1.7.1）。
  - *当指定的路径为**文件夹**时，将会上传指定文件夹下**所有**的文件。*
- 上传文件语法：

```
/upload 本地文件（夹） 目标频道
```

注意事项

`Telegram` 对上传的**单个文件**大小设有**明确限制**：

- **普通用户**：**单个文件**最大上传大小为 `2000 MiB`（约 2 GB）。
- **会员用户（Telegram Premium）**：**单个文件**最大上传大小为 `4000 MiB`（约 4 GB）。

对于Windows用户

上传一个文件

```
/upload C:\files\video.mp4 https://t.me/test
```

上传一个文件夹

版本需要 `≥v1.7.1`。

```
/upload C:\files https://t.me/test
```

对于Linux用户

上传一个文件

```
/upload /home/username/files/video.mp4 https://t.me/test
```

上传一个文件夹

版本需要 `≥v1.7.1` 。

```
/upload /home/username/files https://t.me/test
```

/upload_r

> ⚠️ **版本变更说明：**

版本	说明
`≥v1.8.4`	`/upload_r` 命令支持 `me` 、 `self` 作为目标频道的参数（ `me` 、 `self` 作为目标频道参时，代表指向个人收藏夹 `Saved Messages` ）。

- `/upload_r` 命令是 `/upload` 命令的扩展版本，专为**批量文件上传**场景设计，支持递归处理目录结构，与 `/upload` 不同的是：
  - `/upload_r` 命令接收**文件夹路径**作为其第二个参数，自动**遍历该目录**及其**所有子目录**中的文件。

    ```
    /uploadr C:\files https://t.me/test
    ```

  - 当**第二个参数**为**文件路径**时，自动切换为 `/upload` 的单文件上传模式。

    ```
    /uploadr C:\files\video.mp4 https://t.me/test
    ```

    ```
    /upload C:\files\video.mp4 https://t.me/test
    ```

    以上两个命令在功能上完全等效，系统会自动识别文件类型并采用相应的上传策略。

/download_chat

- `/download_chat` 下载指定频道。
  - 与 `/download` 不同的是： `/download_chat` 支持通过机器人发送的内联键盘进行自定义内容过滤。
  - 该命令支持下载无法复制具体请求链接的对话。
  - 使用该命令后，**需要通过操作机器人回复中的内联键盘**，来设置过滤器、执行任务或取消任务。
  - 需要注意的是，在上一个 `/download_chat` 命令任务未执行或取消前，无法发起新的 `/download_chat` 命令来创建下载任务。
  - 自版本 `≥v1.7.5` 起， `/download_chat` 命令创建的下载任务过滤条件将完全遵循用户在内联键盘中的设置，意味着该命令不会遵循配置文件中的任何规则（例如配置文件中的下载文件类型设置）。
- 目前支持的过滤方式：

过滤方式	默认值
日期范围	`第一条消息的发送日期` ～ `最后的一条消息的发生日期`
文件类型	`视频`、`图片`、`文档`、`音频`、`语音`、`动画（GIF）`、`视频笔记`、`实况照片`
匹配关键词	–
包含评论区	`关`

- 下载指定频道语法：

  ```
  /download_chat 频道链接
  ```

  - 发送命令后，可**根据个人需求**设置过滤器（可选）。
  - 在设置完成后，必须**手动点击**"执行任务"或"取消任务"以继续或终止流程，否则该命令将**始终处于等待状态，并阻塞新的`/download_chat`命令**。

- 下载用户与机器人的对话媒体：

  用户与机器人的对话无法复制具体的请求链接，因此该方法用于解决某些用户与机器人对话中禁止用户进行转发、下载的情况。

  - 首先要获取机器人用户名称，在聊天对话框中点击机器人名称查看其用户名，此处假设机器人名称为`@developer_bot`。
  - 拼接机器人的请求链接为`https://t.me/developer_bot`。

  ```
  /download_chat https://t.me/developer_bot
  ```

- 下载个人收藏夹（任选一种命令即可）：

  - 方式1（任选一种命令即可）：

    ```
    /download_chat me
    ```

    ```
    /download_chat self
    ```

  - 方式2：

    - 首先要获取用户名称，此处假设用户名已设定，个人信息中查看用户名为`@developer`。
    - 拼接个人收藏夹的请求链接为`https://t.me/developer`。

    ```
    /download_chat https://t.me/developer
    ```

- 自版本`≥v1.8.8`起，`/download_chat`命令新增通过匹配关键词下载，匹配关键词说明：

  - 该命令会根据消息的`text（文本）`和`caption（标题）`信息，筛选并下载包含所设置关键词的相关对话内容。
  - 在使用该命令后点击[`🔑设置匹配关键词`]此时，机器人将进入接收关键词模式。
  - 关键词可以为一个或多个，当关键词为多个时，以空格分隔，例如：

`我的最爱`

`学习 资料 健康 营养`

- 用户再次发送已添加的关键词时，机器人会提供一个[⌨️内联键盘]按钮用于[🗑️移除]或[🧿忽略]该关键词。
- 同一时段内发送多个相同的已添加关键词，只会对第一个触发移除选项，后续重复内容不会重复触发。
- 在[🔑设置匹配关键词]界面：
  - 点击[✅确认关键词]时，将会添加所有设置的关键词。
  - 点击[❌取消]时，将会清空所有设置的关键词。
  - 点击[🔙返回]时，不做任何操作，仅返回至主页面。

- 包括评论区设置说明：
  - 评论区开关：
    - 默认为关闭状态。
    - 开启时，按钮状态为[✅包含评论区]，匹配到的消息，存在评论区模块时，进一步匹配评论区。
    - 关闭时，按钮状态为[❌包含评论区]，匹配到的消息，存在评论区模块时，忽略评论区内容。
  - 评论区功能适用性：
    - 平台部分频道在消息下方设有评论区模块，该功能为可选扩展项。用户需基于个人实际需求，自行判断是否需同步获取评论区内容。评论区匹配与过滤规则：
    - 开启包含评论区选项后，即便消息因`媒体类型`被过滤而未下载，但**只要该消息存在评论区**，其评论同样会被获取（`≥v2.0.8`）。
    - **评论区的过滤条件仅遵循本次任务中用户指定的`媒体类型`作为过滤条件，其余过滤条件将被忽略（例如日期）。**
  - 任务处理时效性：
    - 开启包含评论区选项后，需对所匹配到的消息的评论区进行读取，消息过多时，任务创建至完成的时间可能相应延长，需耐心等待。

2.3.配置文件说明

用户配置文件

> [!NOTE]
> 用户配置文件通常无需自行配置，此处旨在介绍全局配置文件中各参数的含义。
> 用户配置文件会在软件启动时自动在软件目录下生成`config.yaml`文件，并详细地引导用户配置参数。
> **手动编辑**`config.yaml`对配置进行修改时，请仔细阅读下面内容理解各参数含义。
> 需注意配置文件使用的引号、冒号均为半角，冒号后需有一个空格。

```yaml
api_hash: xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
api_id: 'xxxxxxxx'
bot_token: 123456:ABC-DEF1234ghIkl-zyx57W2v1u123ew11
download_type:
  - video
  - photo
  - document
  - audio
  - voice
  - animation
  - video_note
  - live_photo
is_shutdown: true
links: D:\path\where\your\link\files\save\content.txt
max_retries:
  download: 5
  upload: 3
max_tasks:
  download: 3
  upload: 3
proxy:
  enable_proxy: true
  hostname: 127.0.0.1
  scheme: socks5
  port: 10808
  username: null
  password: null
save_directory: F:\directory\media\where\you\save
session_directory: F:\directory\session\where\you\save
temp_directory: F:\directory\temp\where\you\save
```

参数说明

参数	说明
`api_hash`	申请的`api_hash`（Telegram API 凭据）。
`api_id`	申请的`api_id`（Telegram API 凭据，需加引号）。
`bot_token`	机器人令牌（选填）；不填则无法使用机器人功能，可前往BotFather免费申请。
`download_type`	需要下载的媒体类型，详见`download_type`参数支持下载的媒体类型表。
`is_shutdown`	下载完成后是否自动关机，支持`true`、`false`。
`links`	链接地址文本文件路径（一个链接一行，不要加引号，运行前准备好）。
`max_retries`	任务重试次数，含`download`（下载任务重试次数）与`upload`（上传任务重试次数）。
`max_tasks`	最大并发任务数，含`download`与`upload`。
`proxy`	代理配置，子字段详见`proxy`参数配置项表；不使用代理时请将`enable_proxy`设为`false`（注意键值间冒号后需保留空格，否则`YAML`解析报错）。
`save_directory`	下载媒体保存目录（支持通配符）。
`session_directory`	会话保存目录（支持通配符）。
`temp_directory`	缓存保存目录（支持通配符）。

`download_type`参数支持下载的媒体类型

下载类型	说明
`video`	视频。
`photo`	图片。
`document`	文档。
`audio`	音频。
`voice`	语音。
`animation`	动画（GIF）。
`video_note`	视频笔记。
`live_photo`	实况照片（未指定且`photo`存在时视为`photo`）。

`proxy`参数配置项

配置项	说明
`enable_proxy`	是否启用代理，支持`true`、`false`。
`scheme`	代理协议，如`socks5`、`http`。
`hostname`	代理服务器`IP`或域名。
`port`	代理服务器端口。
`username`	代理认证用户名（选填）。
`password`	代理认证密码（选填）。

通配符说明

自版本`≥v1.7.4`起，`save_directory`参数将支持通配符，通配符允许用户动态生成存储路径。系统会在下载时自动将通配符替换为对应的实际值，实现按规则自动分类存储。

- 目前`save_directory`支持的通配符如下表所示：

通配符	意义
`%CHAT_ID%`	以实际`频道ID`作为指定路径填充。
`%CHAT_NAME%`	以实际`频道名`作为指定路径填充（≥v1.9.6）。
`%MIME_TYPE%`	以实际`文件类型`作为指定路径填充。

- 用法示例1：

```
F:\directory\media\%CHAT_ID%
```

- 用法示例2（≥v1.9.6）：

```
F:\directory\media\%CHAT_NAME%
```

- 用法示例3：

```
F:\directory\media\%MIME_TYPE%
```

- 用法示例4：

```
F:\directory\media\%CHAT_NAME%\%CHAT_ID%\%MIME_TYPE%
```

全局配置文件

> [!NOTE]
> 全局配置文件通常无需自行配置，此处旨在介绍全局配置文件中各参数的含义。
> 部分参数支持通过机器人设置进行修改。

```yaml
console_log_level: WARNING
export_table:
  count: false
  link: false
file_log_level: INFO
forward_type:
  animation: true
  audio: true
  document: true
  photo: true
  text: true
  video: true
  voice: true
  video_note: true
  live_photo: true
notice: false
upload:
  delete: false
  download_upload: true
```

参数说明

参数	说明
`console_log_level`	终端显示的最低日志级别（如`WARNING`）。
`export_table`	运行结束时是否导出统计表，含`count`（下载计数统计表）与`link`（下载链接统计表）。
`file_log_level`	`TRMD_LOG.log`文件中记录的最低日志级别（如`INFO`）。
`forward_type`	控制`/listen_forward`与`/forward`命令可转发的文件类型，详见`forward_type`参数支持转发的媒体类型。
`notice`	机器人启动时是否发送启动通知。
`upload`	受限内容处理，含`delete`（下载上传完成后是否删除本地文件）与`download_upload`（是否下载后上传到指定转发频道）。

`forward_type`参数支持转发的媒体类型

下载类型	说明
`text`	文本。
`video`	视频。
`photo`	图片。
`document`	文档。
`audio`	音频。
`voice`	语音。
`animation`	动画（GIF）。
`video_note`	视频笔记。
`live_photo`	实况照片（未指定且`photo`存在时视为`photo`）。

全局配置文件存放路径

对于Windows用户

```
%APPDATA%/TRMD/.CONFIG.yaml
```

对于Linux用户

```
~/.config/TRMD/.CONFIG.yaml
```

2.4.使用注意事项

▼ **点击展开**

链接获取方法

对想要保存的媒体文件点击**鼠标右键**然后选择**复制消息直链**，如下图所示：

---

links参数的文本内容具体写法

如下图所示，即一行代表一个链接：

常见的links参数的文本内容的错误写法

如下图所示：

关于私密频道链接的下载行为

- 要求当前所登录的`Telegram`账号，在这个私密频道中，否则会报错无法下载。
- 这是`Telegram`服务端设计所决定的，任何下载器都无法越界。

重复链接问题说明

问题描述

- 如上图所示，在提交的下载任务中，存在多个**前缀相同但参数不同**的链接（如`?comment`、`?single`或`?single&comment`）。

问题原因

- 当链接包含`?comment`参数时，会自动下载**原始消息及其评论区内容**。
- 如果同时提交**相同前缀但无`?comment`的链接**，会导致**同一资源被重复添加**至下载队列。
- 若前一次任务尚未完成，重复提交相同资源会触发**任务冲突**，进而引发下载异常。

解决方案

- **`≥v2.0.8`已针对重复链接导致的资源竞争问题进行自动处理。**
- **仅需提交一个完整链接**（如带`?comment`的版本），系统会自动处理**原始内容及评论区**，无需额外提交无参数版本。
- **避免重复提交相同资源**，确保每条链接的`t.me/c/<频道>/<消息ID>`部分唯一，防止任务冗余。
- 由于这些链接的**频道名**和**消息ID**完全一致，实际上指向的是**同一资源**的不同表现形式。
- 但请注意以下参数的功能差异：
  - `?comment`用于标识是否下载**评论区**内容。
  - `?single`用于在**媒体组**中指定仅下载该消息ID对应的单个资源。
  - `?single&comment`用于指定仅下载评论区中该消息ID所对应的内容。

`Telegram`字段解释

字段	解释
`?comment`	**评论区**的链接。
`?single`	**单独**的链接。
`?single&comment`	**评论区**中**单独**的链接。
`/c`	**私密**频道的链接。
`https://t.me/TEST/111/666`	频道`TEST`**话题**`111`的链接。
`https://t.me/c/1111111111/333/666`	**私密**频道`1111111111`**话题**`333`的链接。

链接解释说明

`Telegram`**链接组成**

频道类型	链接组成
正常频道	`https://t.me/频道名/消息ID`
私密频道	`https://t.me/c/频道名(10位纯数字)/消息ID`
话题频道	`https://t.me/频道名/话题ID/消息ID`
私密话题频道	`https://t.me/c/频道名(10位纯数字)/话题ID/消息ID`

`Telegram`链接所有链接格式

"所有"指的是如果有**合并发送为一组**的文件，则给定一个链接，所有**合并发送的文件**会被全部下载。

"媒体"指的是用户在`config.yaml`文件中，通过`download_type`参数所设置的所需下载类型的媒体。

链接	实际频道名	消息ID	解释
`https://t.me/TEST/111`	`TEST`	`111`	下载该链接的**所有媒体**。
`https://t.me/TEST/111?single`	`TEST`	`111`	下载该链接的**对应的一个媒体**。
`https://t.me/TEST/111?comment=666`	`TEST`	`111`	下载该链接的**所有媒体**的同时，下载该链接下方**评论区里所有的媒体（前提该链接存在评论区，否则视为`https://t.me/TEST/111`处理）**。
`https://t.me/TEST/111?single&comment=666`	`TEST`	`111`	下载该链接下方的**评论区**的**对应的一个媒体**。
`https://t.me/c/1111111111/666`	`-1001111111111`	`666`	下载该**私密频道**链接的**所有**媒体。
`https://t.me/TEST/111/666`	`TEST`	`666`	下载该**话题**链接的**所有**媒体。
`https://t.me/c/1111111111/333/666`	`-1001111111111`	`666`	下载该**私密话题**链接的所有媒体。

如何下载评论区的文件

复制的链接时，带`?comment`字段

通常在评论区复制任意一条消息链接即可。

评论区链接的下载行为规则的说明

标准链接（无`?comment`参数）

- 仅下载**消息正文内容**（即频道/群组中直接发布的原始消息）。
- **不包含评论区内容**，即使原消息存在评论，也不会被纳入下载任务。

带`?comment`参数的链接

- 下载**消息正文 + 关联的全部评论区内容**（完整会话结构）。
- 若原消息**无评论区**（如频道消息或评论功能关闭），则**仅下载消息正文内容**，与无参数版本行为一致。

非下载评论区的推荐写法

如下表所示:

频道类型	链接
正常频道	https://t.me/xxx/111
私密频道	https://t.me/c/xxxxxxxxxx/111
话题频道	https://t.me/xxx/xxx/111
私密话题频道	https://t.me/c/xxxxxxxxxx/xxx/111

关于`?single`及`?single&comment`参数的下载行为说明

功能变更概述

自`≥v1.5.8`版本起，链接中包含`?single`或`?single&comment`参数时，系统将启用**单文件下载模式**。此模式专为以下场景设计与优化：

- 解决`≥1.5.8`版本`/listen_download`当监听到合并发送的文件时，出现**重复下载问题**。
- 用户需求，仅需从合并发送的多媒体组中提取**特定单一文件**。
- 用户需求，仅需下载评论区中的**单个指定媒体**（避免评论区媒体过多时，迟迟下载不到想要的文件）。

参数行为详解

参数格式	下载范围	应用场景
`xx?single`	仅下载消息正文中的**xx对应媒体文件**	从合并图组/视频组提取单文件
`?single&comment=xx`	仅下载评论区中的**xx所对应的媒体文件**	获取评论区单独分享的图片/视频

版本兼容性说明

- 此特性**仅对**`≥v1.5.8`**版本**生效。
- 历史版本中这些参数可能被忽略，导致完整内容下载。

最佳实践建议

单一文件下载

当消息包含多个媒体文件时，使用标准链接附加`?single`参数可精准获取首个文件：

```
https://t.me/c/123456789/123?single
```

评论区单文件下载

需从评论区单独下载文件时，应采用复合参数格式：

```
https://t.me/c/123456789/123?single&comment=xx
```

参数互斥原则

- 避免同时提交同一消息的完整版和单文件版链接（`≥v2.0.8`已针对重复链接导致的资源竞争问题进行自动处理）。
- 单文件模式与评论区下载模式（`?comment`）不可混用。

2.5.软件更新教程

3.0.在生产环境中运行

***推荐**使用`Python==3.13.2`作为该项目环境（避免使用其他`Python`版本导致运行时出现报错）。*

对于Windows用户

*需自行安装`python`与`git`并配置**环境变量**。*

```
git clone https://github.com/Gentlesprite/Telegram_Restricted_Media_Downloader.git
cd Telegram_Restricted_Media_Downloader
python -m pip install --upgrade pip
pip install -r requirements.txt
python main.py
```

对于Linux用户

*克隆本项目并**进入项目目录**。*

```
git clone https://github.com/Gentlesprite/Telegram_Restricted_Media_Downloader.git
cd Telegram_Restricted_Media_Downloader
```

***更新**`pip`版本（推荐）。*

```
python3 -m pip install --upgrade pip
```

*创建并使用**虚拟环境**（可选）。*

```
python3 -m venv venv
source venv/bin/activate
```

*安装程序运行**所需依赖**并等待全部安装完成。*

```
pip3 install -r requirements.txt
```

运行程序（到这一步就安装完成并运行了，后面是注意事项）。

```
python3 main.py
```

注意事项

如果选择创建虚拟环境运行，在下次运行时也需要先激活虚拟环境。

在项目目录下，**激活虚拟环境**后再运行程序。

```
source venv/bin/activate
python3 main.py
```

*如果提示**没有安装pip**使用如下命令进行安装：*

```
sudo apt update
sudo apt-get install python3-pip
```

关于更新

*在**项目目录**下打开终端使用如下命令拉取仓库当前的**最新版本**：*

```
git pull
```

*由于新版本可能使用了**新的依赖**，使用`git pull`拉取后，最好更新一下依赖（如果是**虚拟环境**请先激活再执行）。*

```
pip3 install -r requirements.txt
```

4.0.运行前设置命令行参数（高阶用法）

> [!NOTE]
> 自版本`≥v1.8.3`起：
> 用户可在**运行前**通过**命令行参数**对**软件运行配置**进行**更多自定义设置**。

▼ **点击展开**

***设置命令行运行参数**需先在**软件目录**打开**终端**，或**任意位置**打开终端**进入软件目录**（**经常**使用建议**配置环境变量**）。*

目前支持的命令行参数用法及解释如下表所示：

短参数	长参数	解释
`-h`	`--help`	展示帮助
`-v`	`--version`	展示版本信息
`-q`	`--quiet`	跳过重新配置文件的确认提示
`-c`	`--config`	设置用户配置文件的路径
`-s`	`--session`	设置会话文件的路径
`-t`	`--temp`	设置运行缓存的路径
`-p`	`--port`	设置网页面板的端口

长参数与短参数最终结果一致。

4.1.参数

-h、–help

该参数用于展示帮助。

- 对于生产环境用户（**需要先完成前置步骤***"3.0.在生产环境中运行"*）：

```
python3 main.py -h
```

```
python3 main.py --help
```

- 对于Windows用户：

```
TRMD.exe -h
```

```
TRMD.exe --help
```

- 对于Linux用户：

```
./TRMD -h
```

```
./TRMD --help
```

-v、–version

该参数用于展示版本信息。

- 对于生产环境用户（**需要先完成前置步骤***"3.0.在生产环境中运行"*）：

```
TRMD.exe -v
```

```
TRMD.exe --version
```

- 对于Windows用户：

```
TRMD.exe -v
```

```
TRMD.exe --version
```

- 对于Linux用户：

```
./TRMD -v
```

```
./TRMD --version
```

-q、–quiet

使用须知
1.该参数用于跳过重新配置文件的确认提示。
2.用户配置文件缺少必要参数时，指定该参数无效。
3.该参数为一次性设置，不记忆。

- 对于生产环境用户（**需要先完成前置步骤***"3.0.在生产环境中运行"*）：

```
python3 main.py -q
```

```
python3 main.py --quiet
```

- 对于Windows用户：

```
TRMD.exe -q
```

```
TRMD.exe --quiet
```

- 对于Linux用户：

```
./TRMD -q
```

```
./TRMD --quiet
```

-c、–config

使用须知
1.该参数用于设置用户配置文件的路径。
*2.**该参数旨在解决多个实例（多开）场景下，避免重复部署软件本体而设计的配置分离方案。***
*3.该参数需指定一个**符合**"2.3.配置文件说明（用户配置文件）"格式规范的文件，**并且后缀名需为**`.yaml`。*
*4.当指定的**文件路径无效**时，将使用软件**默认**设置。*
5.该参数为一次性设置，不记忆。

- 对于生产环境用户（**需要先完成前置步骤**"3.0.在生产环境中运行"）：

  以`Linux`系统为例（`Winodws`系统同理），此处假设用户配置文件位于`/home/username/files/example.yaml`。

  ```
  python3 main.py -c /home/username/files/example.yaml
  ```

  ```
  python3 main.py --config /home/username/files/example.yaml
  ```

- 对于Windows用户：

  此处假设用户配置文件位于`C:\files\example.yaml`。

  ```
  TRMD.exe -c C:\files\example.yaml
  ```

  ```
  TRMD.exe --config C:\files\example.yaml
  ```

- 对于Linux用户：

  此处假设用户配置文件位于`/home/username/files/example.yaml`。

  ```
  ./TRMD -c /home/username/files/example.yaml
  ```

  ```
  ./TRMD --config /home/username/files/example.yaml
  ```

-s、–session

> ⚠️ **版本变更说明：**

版本	说明
`≥v1.8.5`	`-s`、`--session`参数在设置后将被保存到用户配置文件的`session_directory`参数中，下次使用时无需重复指定，除非需要修改该设置。

使用须知
1.该参数用于设置会话文件的路径。
2.软件会在用户登录成功时后，默认在运行目录下生成`session`文件夹，用于保存当前账号信息，以便后续快速登录。
*3.**该参数旨在**解决多账号登录场景下需**手动重命名**会话文件的问题，使用该参数用户可通过**直接**指定不同路径，选择对应账号进行登录。*
*4.该参数需指定一个**文件夹**，可包含已有的`.session`文件，指定为空文件夹或不存在时，登录后将在**该路径自动生成**会话文件。*
5.该参数设置后会被记忆，下次无需重复设置（`≥v1.8.5`）。

- 对于生产环境用户（**需要先完成前置步骤**"*"3.0.在生产环境中运行"*）：

  以`Linux`系统为例（`Winodws`系统同理），此处假设会话文件位于`/home/username/files/session`。

  ```
  python3 main.py -s /home/username/files/session
  ```

  ```
  python3 main.py --session /home/username/session
  ```

- 对于Windows用户：

  此处假设会话文件位于`C:\files\session`。

  ```
  TRMD.exe -s C:\files\session
  ```

  ```
  TRMD.exe --session C:\files\session
  ```

- 对于Linux用户：

  此处假设会话文件位于`/home/username/files/session`。

  ```
  ./TRMD -s /home/username/files/session
  ```

  ```
  ./TRMD --session /home/username/files/session
  ```

-t、–temp

> ⚠️ **版本变更说明：**

版本	说明
`≥v1.8.5`	`-t`、`--temp`参数在设置后将被保存到用户配置文件的`temp_directory`参数中，下次使用时无需重复指定，除非需要修改该设置。

使用须知
1.该参数用于设置运行缓存的路径。
*2.缓存即软件运行过程中，记录**下载和上传**信息的**存储中转站**。*
*3.**断点续传**的实现同样离不开缓存机制的设计，用户可**根据实际需求自定义缓存存放的路径**。*
*4.该参数需指定一个**文件夹**，指定为空文件夹或不存在时，运行时将在**该路径自动生成**生成缓存文件。*
5.该参数设置后会被记忆，下次无需重复设置（`≥v1.8.5`）。

- 对于生产环境用户（**需要先完成前置步骤***"3.0.在生产环境中运行"*）：

  以`Linux`系统为例（`Winodws`系统同理），此处假设缓存文件位于`/home/username/files/temp`。

  ```
  python3 main.py -t /home/username/files/temp
  ```

  ```
  python3 main.py --temp /home/username/temp
  ```

- 对于Windows用户：

  此处假设缓存文件位于`C:\files\temp`。

  ```
  TRMD.exe -t C:\files\temp
  ```

  ```
  TRMD.exe --temp C:\files\temp
  ```

- 对于Linux用户：

  此处假设缓存文件位于`/home/username/files/temp`。

  ```
  ./TRMD -t /home/username/files/temp
  ```

  ```
  ./TRMD --temp /home/username/files/temp
  ```

-p、–port

使用须知
1.网页面板默认启用，通过浏览器查看下载进度。
2.网页面板会在后台启动网页服务，终端仍可正常操作。
3.`Web配置`信息会在运行的终端提供，以便打开网页时输入账号密码。
*4.**关闭浏览器窗口不会影响程序运行**，程序退出时面板自动关闭。*
*5.该参数用于指定网页面板的**端口号**，范围为`0`~`65535`，若不指定，将使用`2921`端口，端口被占用时自动分配。*
6.该参数为一次性设置，不记忆。

- 对于生产环境用户（**需要先完成前置步骤**"*"3.0.在生产环境中运行"*）：

  此处假设使用`1024`端口。

  ```
  python3 main.py -p 1024
  ```

  ```
  python3 main.py --port 1024
  ```

- 对于Windows用户：

  此处假设使用`1024`端口。

  ```
  TRMD.exe -p 1024
  ```

  ```
  TRMD.exe --port 1024
  ```

- 对于Linux用户：

  此处假设使用`1024`端口。

  ```
  ./TRMD -p 1024
  ```

  ```
  ./TRMD --port 1024
  ```

5.0.通过编译后运行

***推荐**使用`Python==3.13.2`作为该项目环境，并使用**虚拟环境**，**避免**添加不必要的库，从而**减小**输出的文件大小。*

`Windows`*构建说明：*

- **Windows构建会启用`--clang`**。
- 需安装`Visual Studio`并勾选`C++ Clang Compiler for Windows`组件以提供`clang-cl`。

- 下载`Visual Studio`时须留意版本，其安装的`LLVM`/`clang`版本须≥19，因Nuitka编译所用的`#embed`（C23）常量嵌入仅在该版本起受支持。
- 在`Windows`下，`build.py`必须在**x64 Native Tools Command Prompt for VS**中运行，`Nuitka`依靠该命令行预置的`MSVC`与`Windows SDK`编译环境。
- 在`CMD`/`PowerShell`中运行，将因所需环境变量在编译期报错，并且`Nuitka`不再回退至`gcc`/`zig`（仅当`MSVC`完全探测不到时才触发回退）。

同*"3.0.在生产环境中运行"*前置步骤一致。

执行代码：

```
python build.py
```

6.0.通过Docker运行

Docker环境运行配置文件模板参考（前提：需完全按照后续教程提供的命令启动Docker）：

```yaml
api_hash: xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
api_id: 'xxxxxxxx'
bot_token: 123456:ABC-DEF1234ghIkl-zyx57W2v1u123ew11
download_type:
- video
- photo
- document
- audio
- voice
- animation
- video_note
- live_photo
is_shutdown: false
links: /app/TRMD/links.txt
max_retries:
  download: 5
  upload: 3
max_tasks:
  download: 3
  upload: 3
proxy:
  enable_proxy: true
  hostname: 192.168.1.10
  scheme: socks5
  port: 10808
  username: null
  password: null
save_directory: /app/downloads/%CHAT_NAME%/%MIME_TYPE%
session_directory: /app/sessions
temp_directory: /app/temp
```

由于`Docker`容器的工作目录为`/app`，配置文件中的路径应填写**容器内路径**，实际文件通过`docker run`的`-v`卷挂载映射至宿主机目录：

配置项	容器内路径（写入配置）	对应的主机路径
`save_directory`	`/app/downloads/%CHAT_NAME%/%MIME_TYPE%`	`./downloads/%CHAT_NAME%/%MIME_TYPE%`
`session_directory`	`/app/sessions`	`./sessions`
`temp_directory`	`/app/temp`	`./temp`
`links`	`/app/TRMD/links.txt`	`./config/links.txt`

- *另挂载`./form:/app/form`为程序内部统计/表单导出目录（`/app/form`）。
- *代理`hostname`（示例`192.168.1.10`）仅作为参考，实际部署时需替换为各自代理服务器的`ip`地址（可通过`ip addr`或`ifconfig`查看本机局域网`IP`）。若运行环境已可直接访问`Telegram`服务器，则可不启用代理：将`enable_proxy`设为`false`。

方式1：

- *确保`docker`已安装并配置**环境变量**。
- 使用`docker`从远程仓库拉取镜像，注意`TRMD`版本需要`≥2.0.2`：

```
docker pull gentlesprite/telegram_restricted_media_downloader:latest
```

- 首次使用时需要进入容器配置文件：

```
docker run -it --rm \
  --name trmd \
  -v ./config:/app/TRMD \
  -v ./sessions:/app/sessions \
  -v ./downloads:/app/downloads \
  -v ./temp:/app/temp \
  -v ./form:/app/form \
  -p 2921:2921 \
  -e TZ=Asia/Shanghai \
  gentlesprite/telegram_restricted_media_downloader:latest \
  python main.py --config /app/TRMD/config.yaml --port 2921
```

- 如果是通过`Windows`使用`wsl2`运行`docker`：

```
docker run -it --rm --name trmd -v ./config:/app/TRMD -v ./sessions:/app/sessions -v ./downloads:/app/downloads -v ./temp:/app/temp -v ./form:/app/form -e TZ=Asia/Shanghai gentlesprite/telegram_restricted_media_downloader:latest python main.py --config /app/TRMD/config.yaml --port 2921
```

- 如果已配置完成的情况下，直接创建并启动容器：

```
docker run -it -d \
  --name trmd \
  -v ./config:/app/TRMD \
  -v ./sessions:/app/sessions \
  -v ./downloads:/app/downloads \
  -v ./temp:/app/temp \
  -v ./form:/app/form \
  -p 2921:2921 \
  -e TZ=Asia/Shanghai \
  gentlesprite/telegram_restricted_media_downloader:latest \
  python main.py --config /app/TRMD/config.yaml --port 2921
```

- 如果是通过`Windows`使用`wsl2`运行`docker`：

```
docker run -it -d --name trmd -v ./config:/app/TRMD -v ./sessions:/app/sessions -v ./downloads:/app/downloads -v ./temp:/app/temp -v ./form:/app/form -p 2921:2921 -e TZ=Asia/Shanghai gentlesprite/telegram_restricted_media_downloader:latest python main.py --config /app/TRMD/config.yaml --port 2921
```

方式2：

- *确保`git`、`docker`、`docker-compose`已安装并配置**环境变量**。*
- 使用`git`克隆仓库：

```
git clone https://github.com/Gentlesprite/Telegram_Restricted_Media_Downloader.git
```

- 进入项目文件夹：

```
cd Telegram_Restricted_Media_Downloader
```

- 首次使用时需要进入容器配置文件：

```
docker-compose run --rm --service-ports --name trmd trmd
```

- 如果已配置完成的情况下，直接创建并启动容器：

```
docker-compose up -d
```

以上两种方式启动后，通过在终端或在`Telegram`中向机器人发送`/help`命令查看`Web配置`以访问网页面板（包含端口、账号、密码与访问链接），类似下表：

属性	内容
端口	`2921`
账号	`cLJqKG3b`
密码	`AiJaKSObcRCZ`
链接	`http://127.0.0.1:2921`

账号密码由系统随机生成，使用浏览器打开http://127.0.0.1:2921网页输入账号密码即可进入。

- **仅在已配置机器人的情况下**，也可在`Telegram`中向机器人发送`/help`命令查看上述`Web配置`。
- 网页面板默认使用`2921`端口，可通过`-p`、`--port`参数指定自定义端口，端口被占用时自动分配可用端口。
- 网页面板在后台启动网页服务，终端仍可正常操作；**关闭浏览器窗口不会影响程序运行**，程序退出时面板自动关闭。
- 首次通过账号密码登录成功后，面板会向**此浏览器**下发一个有效期`30`天的`Cookie`，使用此浏览器的访问行为在软件重启或重新访问后依然免密。
- 如需远程访问，请自行查看并替换`ip`地址为公网`ip`，并开放`2921`或`自定义`的端口。

其他命令：

- 如需查看运行日志：

  ```
  docker logs trmd
  ```

- **不再使用**时，停止并删除容器：

  ```
  docker stop trmd && docker rm trmd
  ```

7.0.联系作者

Telegram：@Gentlesprite

邮箱：Gentlesprites@outlook.com

8.0.支持作者


支付宝
推荐使用支付宝
本伟 (**宇)
打开支付宝[扫一扫]
申请官方收钱码：拨打 95188-6
推荐使用微信支付
饿不死盘神 (**宇)
微信支付